

Day 28 & 29: Apache Airflow (Fundamentals & Advanced)

1. Curriculum Concepts & Explanations

Day 28: Airflow Fundamentals

- **DAGs (Directed Acyclic Graphs):** The core concept of Airflow. A DAG represents a workflow where nodes are tasks and directed edges represent dependencies. "Acyclic" means the workflow flows in one direction and never loops back on itself.
- **Operators:** The building blocks of Airflow. While a DAG defines *how* tasks run, an Operator defines *what* a single task does. Common examples include `PythonOperator`, `BashOperator`, and `SparkSubmitOperator`.
- **Scheduling:** Airflow triggers DAGs based on a schedule, defined using standard CRON expressions (e.g., `'0 0 * * *'` for daily at midnight) or `timedelta` objects.
- **Dependencies & Task Management:** Task execution order is controlled using bitshift operators (`>>` for downstream, `<<` for upstream). For example, `task_1 >> task_2` means task 2 waits for task 1 to succeed.

Day 29: Airflow Advanced

- **Sensors:** A special type of Operator that waits for an external event to occur before advancing. For instance, a `FileSensor` waits for a file to land in a specific directory.
 - **XComs (Cross-Communication):** Since tasks run independently (often on different worker nodes), XComs allow tasks to exchange small amounts of metadata (like a record count or a generated file path).
 - **Branching:** Using the `BranchPythonOperator`, you can create conditional logic in your DAG (e.g., "If data quality passes, proceed to load; else, trigger an alert task").
 - **Dynamic DAGs & Best Practices:** Generating multiple DAGs dynamically via Python code (e.g., a loop creating a DAG for each client). Best practices include keeping DAG files light (no heavy processing at the root level) and making tasks idempotent.
-

2. Problem Statement: Automated Spark ETL Pipeline Orchestration

Objective: Build an Airflow DAG that orchestrates a batch PySpark ETL job for daily e-commerce sales.

- **Sensor:** Wait for the daily raw CSV file to arrive in a designated folder.
- **Spark Processing:** Use `SparkSubmitOperator` to trigger a PySpark job that cleans the data and aggregates daily sales.
- **XCom & Branching:** The Spark job writes the row count to a text file. A Python task reads this count and pushes it via XCom. A Branching task decides: if rows > 0, proceed to "Success Alert"; if 0, proceed to "Failure Alert".

3. Infrastructure Setup (docker-compose.yml)

This setup brings up Airflow (LocalExecutor) alongside your existing Spark architecture.

```
version: '3.8'
services:
  postgres:
    image: postgres:13
    environment:
      POSTGRES_USER: airflow
      POSTGRES_PASSWORD: airflow
      POSTGRES_DB: airflow

  airflow-webserver:
    image: apache/airflow:2.8.1
    environment:
      - AIRFLOW__CORE__EXECUTOR=LocalExecutor
      - AIRFLOW__CORE__SQL_ALCHEMY_CONN=postgresql+psycopg2://airflow:airflow@postgres/
    airflow
      - AIRFLOW__CORE__LOAD_EXAMPLES=False
    volumes:
      - ./dags:/opt/airflow/dags
      - ./data:/opt/airflow/data
    ports:
      - "8080:8080"
    command: webserver
    depends_on:
      - postgres

  airflow-scheduler:
    image: apache/airflow:2.8.1
    environment:
      - AIRFLOW__CORE__EXECUTOR=LocalExecutor
      - AIRFLOW__CORE__SQL_ALCHEMY_CONN=postgresql+psycopg2://airflow:airflow@postgres/
    airflow
      - AIRFLOW__CORE__LOAD_EXAMPLES=False
    volumes:
      - ./dags:/opt/airflow/dags
      - ./data:/opt/airflow/data
    command: scheduler
    depends_on:
      - postgres

  spark-master:
    image: bitnami/spark:3.5
    environment:
      - SPARK_MODE=master
    ports:
      - "8081:8080"
      - "7077:7077"
    volumes:
      - ./data:/workspace/data
      - ./scripts:/workspace/scripts
```

4. Code Implementation

4.1 The PySpark Job (scripts/sales_etl.py)

This script processes the data and saves the row count for Airflow to read.

```
import sys
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, sum

def run_etl(input_path, output_path, metrics_path):
    spark = SparkSession.builder.appName("SalesETL").getOrCreate()

    # Read raw data
    df = spark.read.csv(input_path, header=True, inferSchema=True)

    # Simple Transformation: Aggregate sales by category
    agg_df = df.groupBy("category").agg(sum("price").alias("total_sales"))

    # Save output
    agg_df.write.mode("overwrite").parquet(output_path)

    # Save row count for Airflow XCom consumption
    count = agg_df.count()
    with open(metrics_path, "w") as f:
        f.write(str(count))

    spark.stop()

if __name__ == "__main__":
    input_file = sys.argv[1]
    output_dir = sys.argv[2]
    metrics_file = sys.argv[3]
    run_etl(input_file, output_dir, metrics_file)
```

4.2 The Airflow DAG (dags/sales_pipeline_dag.py)

This DAG uses Sensors, SparkSubmit, XComs, and Branching.

```

from airflow import DAG
from airflow.sensors.filesystem import FileSensor
from airflow.providers.apache.spark.operators.spark_submit import SparkSubmitOperator
from airflow.operators.python import PythonOperator, BranchPythonOperator
from airflow.operators.empty import EmptyOperator
from datetime import datetime, timedelta
import os

default_args = {
    'owner': 'data_engineer',
    'depends_on_past': False,
    'start_date': datetime(2023, 10, 1),
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
}

def read_metrics_and_push(**kwargs):
    # Read the text file generated by Spark
    metrics_file = '/opt/airflow/data/metrics.txt'
    if os.path.exists(metrics_file):
        with open(metrics_file, 'r') as f:
            count = int(f.read().strip())
        # Push to XCom
        kwargs['ti'].xcom_push(key='record_count', value=count)
    else:
        kwargs['ti'].xcom_push(key='record_count', value=0)

def branch_logic(**kwargs):
    # Pull from XCom
    count = kwargs['ti'].xcom_pull(key='record_count', task_ids='read_metrics_task')
    if count > 0:
        return 'success_alert_task'
    return 'failure_alert_task'

with DAG('daily_sales_spark_etl', default_args=default_args, schedule_interval='@daily',
        catchup=False) as dag:

    # 1. SENSOR: Wait for the raw file to arrive
    wait_for_file = FileSensor(
        task_id='wait_for_sales_file',
        filepath='/opt/airflow/data/raw_sales.csv',
        fs_conn_id='fs_default',
        poke_interval=30,
        timeout=600
    )

    # 2. OPERATOR: Run PySpark Job
    run_spark_job = SparkSubmitOperator(
        task_id='run_spark_etl',

```

```

        application='/workspace/scripts/sales_etl.py',
        conn_id='spark_default',
        application_args=[
            '/workspace/data/raw_sales.csv',
            '/workspace/data/processed_sales',
            '/workspace/data/metrics.txt'
        ]
    )

# 3. XCOM: Read metrics from Spark output
read_metrics = PythonOperator(
    task_id='read_metrics_task',
    python_callable=read_metrics_and_push,
    provide_context=True
)

# 4. BRANCHING: Decide path based on XCom value
branch_task = BranchPythonOperator(
    task_id='check_data_quality',
    python_callable=branch_logic,
    provide_context=True
)

# 5. DUMMY ALERTS (Endpoints)
success_alert = EmptyOperator(task_id='success_alert_task')
failure_alert = EmptyOperator(task_id='failure_alert_task')

# 6. Define Dependencies
wait_for_file >> run_spark_job >> read_metrics >> branch_task
branch_task >> [success_alert, failure_alert]

```